

ForesightX Product and Technical Documentation

Version: 2.0

Team: VizMinds

Competition: CreateX

Date: May 21, 2026

1. Executive Summary

ForesightX is a cinematic decision simulation platform that helps a user explore the possible consequences of an important choice before acting. The user enters a current situation, a decision under consideration, and a domain. The system then produces three future timelines: optimistic, realistic, and pessimistic. Each timeline includes a probability, milestones, final state, and actionable factors.

The product is designed for hackathon judging and live demonstration. Its primary value is clarity under uncertainty. Instead of presenting a static answer, ForesightX shows a visible reasoning pipeline, streams progress in real time, and gives users tools to interrogate each future branch after the simulation completes.

Version 2 expands the original working system with follow-up analysis, milestone drill-down, what-if reruns, shareable result cards, a refined responsive layout, and improved deployment documentation.

2. Product Scope

Core user flow:

1. The user enters a situation, decision, and domain.
2. The simulation pipeline streams visible agent activity.
3. Three future outcomes are shown in a structured results view.
4. The user can ask follow-up questions about the completed simulation.
5. The user can expand individual milestone steps for deeper context.
6. The user can modify one variable and rerun the outcome agents without repeating the full orchestration stage.
7. The user can export a branded PNG summary of the result.

Primary users:

Startup founders, students, product teams, career planners, and decision makers who need to compare possible paths quickly.

Out of scope for the current version:

Persistent cloud history, user accounts, billing, team workspaces, and Supabase-backed data storage.

3. System Architecture

Frontend:

React with Vite provides the interactive user experience. Tailwind CSS implements the design system using custom tokens for color, typography, spacing, and animation. The frontend consumes Server-Sent Events for live updates and stores the last five simulations in localStorage.

Backend:

FastAPI provides the simulation API, streaming endpoints, validation, rate limiting, CORS, Sentry initialization, and an admin health endpoint. The backend is containerized with Docker for deployment to a Hugging Face Docker Space.

Agent pipeline:

1. Orchestrator prepares a structured briefing from the user scenario.
2. Optimist, Realist, and Pessimist agents run in parallel.
3. Synthesizer normalizes probabilities and formats the final response.
4. Analyst handles follow-up questions after the simulation.
5. Milestone expansion provides short explanations for clicked timeline steps.

Data storage:

The current version does not require a database. Session history is stored locally in the browser with a maximum of five entries.

4. Feature Summary

Decision input:

The landing screen includes a starfield background, product wordmark, typewriter tagline, two structured text areas, character counters, and a domain selector.

Live simulation:

The simulation screen shows a visual pipeline with status indicators, progress, and streamed agent text. Desktop uses a graph layout. Mobile uses stacked cards to preserve readability.

Results dashboard:

The desktop results view keeps the page fixed in a single viewport. Outcome cards scroll internally. Mobile stacks the cards and allows page scroll so no controls are hidden.

Follow-up analysis:

Users can ask concise follow-up questions after a simulation. Responses stream into a collapsible panel and are appended as separate analysis blocks.

Milestone drill-down:

Each timeline milestone can be expanded inline. The first click streams a short explanation from the backend. The explanation is cached client-side so repeated opens do not call the API again.

What-if branch:

Each outcome card includes a what-if input. The user changes one variable, and the backend reruns only the three outcome agents plus synthesis. The triggering card shows a modified badge.

Share card:

The results screen can export a branded PNG snapshot with the ForesightX title, input summary, timestamp, domain, and all three outcome cards.

5. API Reference

POST /simulate

Streams the full simulation result through Server-Sent Events.

GET /simulate/stream

Native EventSource-compatible simulation stream used by the frontend.

POST /simulate/followup

Streams a follow-up analysis for a completed simulation. Request body includes a question, full simulation context, and session identifier.

POST /simulate/expand-milestone

Streams a two to three sentence explanation for a selected milestone. Request body includes milestone text, outcome context, and outcome type.

POST /simulate/whatif

Streams a partial rerun using the original briefing plus one changed variable. The orchestrator stage is skipped.

GET /admin/health

Returns uptime, request count, last error timestamp, and Sentry status. Protected by the X-Admin-Key header.

6. Security, Reliability, and Monitoring

Input validation:

All backend request bodies use Pydantic models with length limits and explicit field types.

Rate limiting:

The simulation endpoint is limited to 10 requests per minute per IP. Follow-up is limited to 8 requests per minute. Milestone expansion is limited to 15 requests per minute. What-if reruns are limited to 6 requests per minute. Admin health is limited to 5 requests per minute.

CORS:

The backend reads allowed origins from the ALLOWED_ORIGINS environment variable. Production configuration must use the deployed Vercel URL and must not use a wildcard origin.

Secrets:

No secrets are stored in the repository. Frontend variables use the VITE prefix. Backend secrets are configured through local .env files or Hugging Face Space secrets.

Monitoring:

Sentry is initialized in both frontend and backend. The backend also exposes an admin health route for operational checks.

Failure handling:

The backend returns structured stream errors and uses fallback behavior where possible. The frontend has an error boundary and designed error states.

7. Deployment Guide

Frontend deployment:

The frontend deploys to Vercel from the frontend directory. Required environment variable: VITE_BACKEND_URL. Optional variable: VITE_SENTRY_DSN. After GitHub receives a new commit, Vercel automatically rebuilds and deploys.

Backend deployment:

The backend deploys to the existing public Hugging Face Docker Space. Required secrets are GROQ_API_KEY, GEMINI_API_KEY, ADMIN_API_KEY, and ALLOWED_ORIGINS. Optional secrets include SENTRY_DSN and dedicated Groq keys for individual agents.

Preferred backend reupload command:

```
git subtree push --prefix backend hf main
```

If the Hugging Face Space history has diverged, upload the contents of the backend directory through the Space file UI and wait for the Docker rebuild.

Post-deployment checks:

1. Confirm the Space root endpoint returns service status.
2. Confirm /admin/health works with the admin key.
3. Run one full simulation from the Vercel frontend.
4. Test follow-up, milestone drill-down, what-if rerun, history, and share export.

8. Testing Summary

Local validation completed:

Frontend production build passes with npm run build.

Backend import and syntax validation passes with python -m compileall app.

Milestone expansion and what-if endpoints were smoke tested with local streamed responses.

Recommended final acceptance test:

Use the deployed frontend and backend together. Submit a realistic decision scenario, wait for completion, expand one milestone, ask one follow-up, run one what-if rerun, export a PNG, reload from history, and verify no console errors appear.

9. Maintenance Notes

Keep the backend and frontend .env.example files synchronized with any new configuration variables.

Do not enable Supabase tables until persistence is required. If added later, save migrations under backend/migrations and include Row Level Security policies.

Keep Vercel and Hugging Face secrets separate from the repository.

Before a competition demo, verify provider quotas, Hugging Face build status, Vercel deployment status, and CORS allowed origins.

10. Version 2 Deliverables

Version 2 includes:

Follow-up analysis, milestone drill-down, what-if reruns, share card export, responsive simulation and result screens, improved agent labels, updated favicon, updated API documentation, and Hugging Face reupload guidance.